Top-down Multiplayer Shooter Template

(Netcode for GameObjects, Lobby, Relay)
v1.1.1

HEAVYART
2025

About

Multiplayer project template. Designed as a starter pack: recommended to import package into a new empty project.
Contains 5 scenes, 2 characters (player, enemy), 3 weapons (pistol, rifle, shotgun), 4 power-ups (movement speed, fire rate, weapon accuracy, medkit).
Uses Lobby, Relay multiplayer services and Netcode for GameObjects.

Project structure:
\Content - character models, weapon models, animations, icons, materials
\Prefabs - character prefabs (player, enemy), drop element prefabs, UI, weapons
\Scenes - all game scenes, including demo scene at \Scenes\Demo\Demo.unity
\ScriptableObjects - drop element configs, NetworkPrefabsList (required by Netcode)
\Scripts - all game code

Scenes:
BootScene - scene to start from. Used as a place to initialize settings and network. Loads only once - on game start.
LoadingScene - is the scene displayed while switching between other scenes.
MainMenu - menu, where to start a new game or join existing, set player name, select player color and switch server region.
GameScene - scene with gameplay itself
Demo - demo scene at \Scenes\Demo\Demo.unity

Where to start:
We recommend starting exploration of the template by running Demo scene.
It contains gameplay in single-player mode. Demo scene located at Scenes\Demo\Demo.scene
To run the full version, follow the installation guide below.

Running multiple instances:
To run multiple instances on the same computer, use ParrelSync for editor and command line arguments, for standalone builds: -authProfileName uniqueProfileName, where uniqueProfileName - unique name for every instance.
If you are using ParrelSync, uncomment code at LobbyDataControl, lines 109-117.

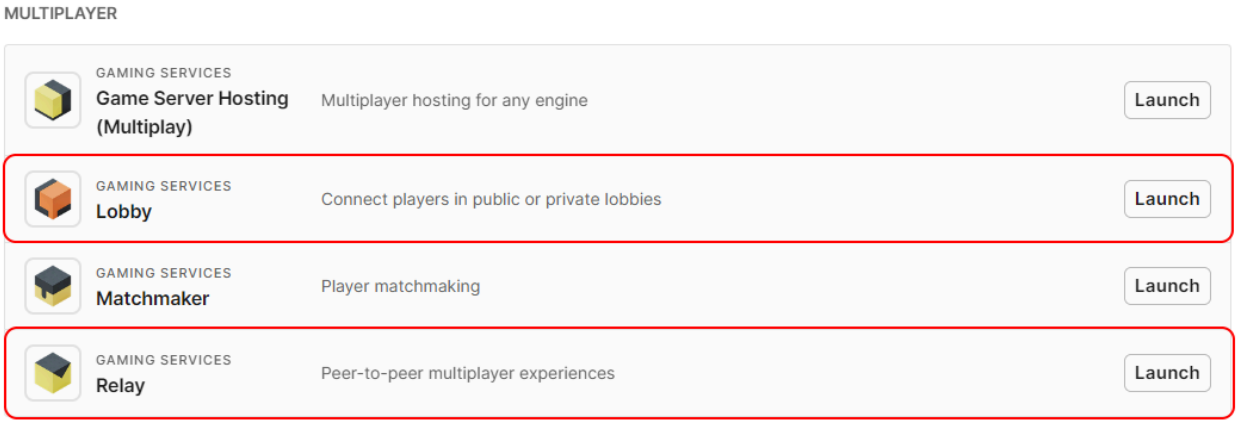
```
108            //Uncomment code below if you are using ParrelSync
109            //#if UNITY_EDITOR
110
111            //if (Application.isEditor && ParrelSync.ClonesManager.IsClone())
112            //{
113            //    string customArgument = ParrelSync.ClonesManager.GetArgument();
114            //    AuthenticationService.Instance.SwitchProfile($"Clone_{customArgument}_Profile");
115            //    PlayerDataKeeper.authProfileName = customArgument;
116            //}
117            //#endif
```

Installation Guide

1. Create project in Unity Cloud Home (if it's not created)
   1.1 Go to Projects, and then select New.
   1.2 Enter project name and specify the COPPA designation.
   1.3 Press Create.

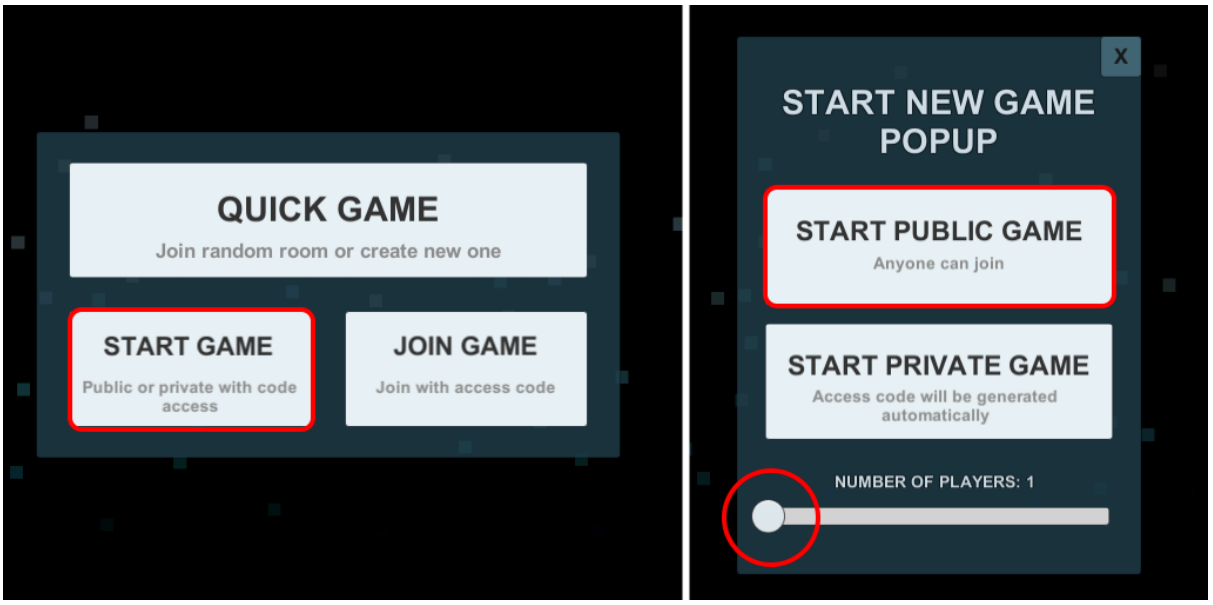
For more information, see the documentation on managing Unity projects.

2. Setup Lobby and Relay in Unity Gaming Services Dashboard > Multiplayer



3. Launch full game version from **\Scenes\BootScene.scene**
or demo version from **\Scenes\Demo\Demo.scene**

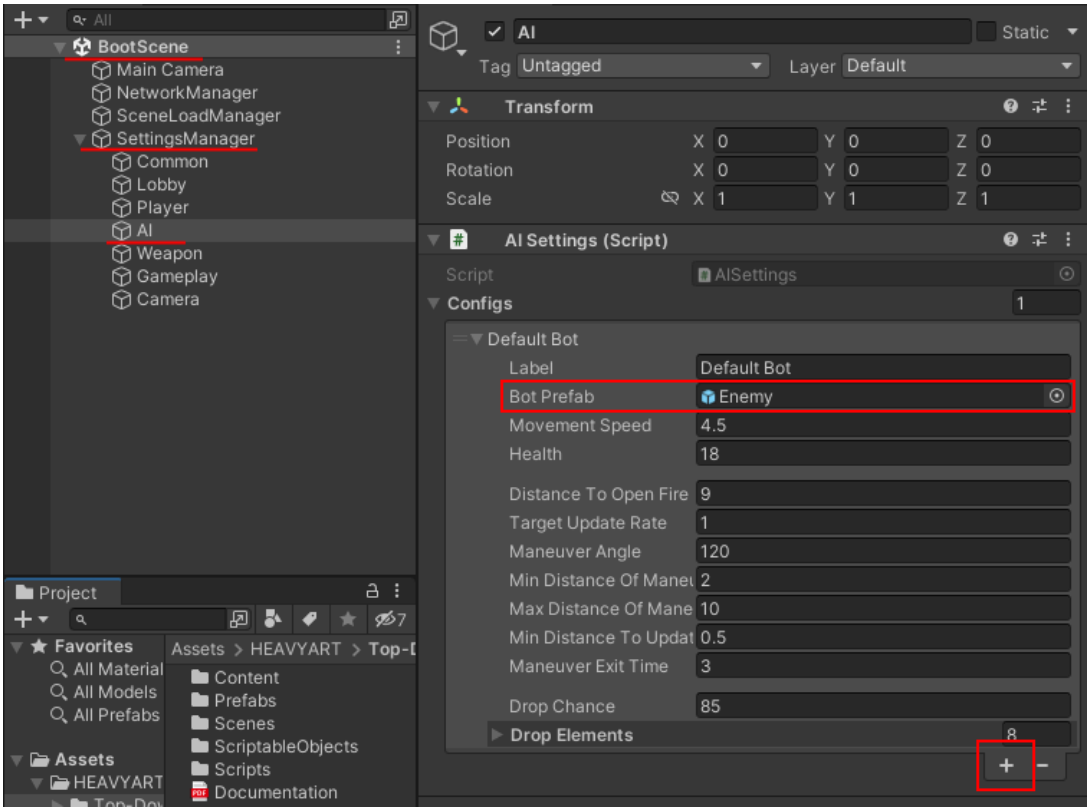
To launch full game for 1 player, press "START GAME",
set "NUMBER OF PLAYERS" to 1 (move slider left) and press "START PUBLIC GAME"



How To

How to add new bot
- Open boot scene.
- Find SettingsManager. Select AI child object.
- Press **+** below Configs collection. It will automatically copy last bot config and add it to the end of Configs collection. Now, it's possible to change its settings.

To add new AI prefab:
- Copy existing one and put its copy to Bot Prefab field.
- Add new prefab to NetworkPrefabsList located at \ScriptableObjects



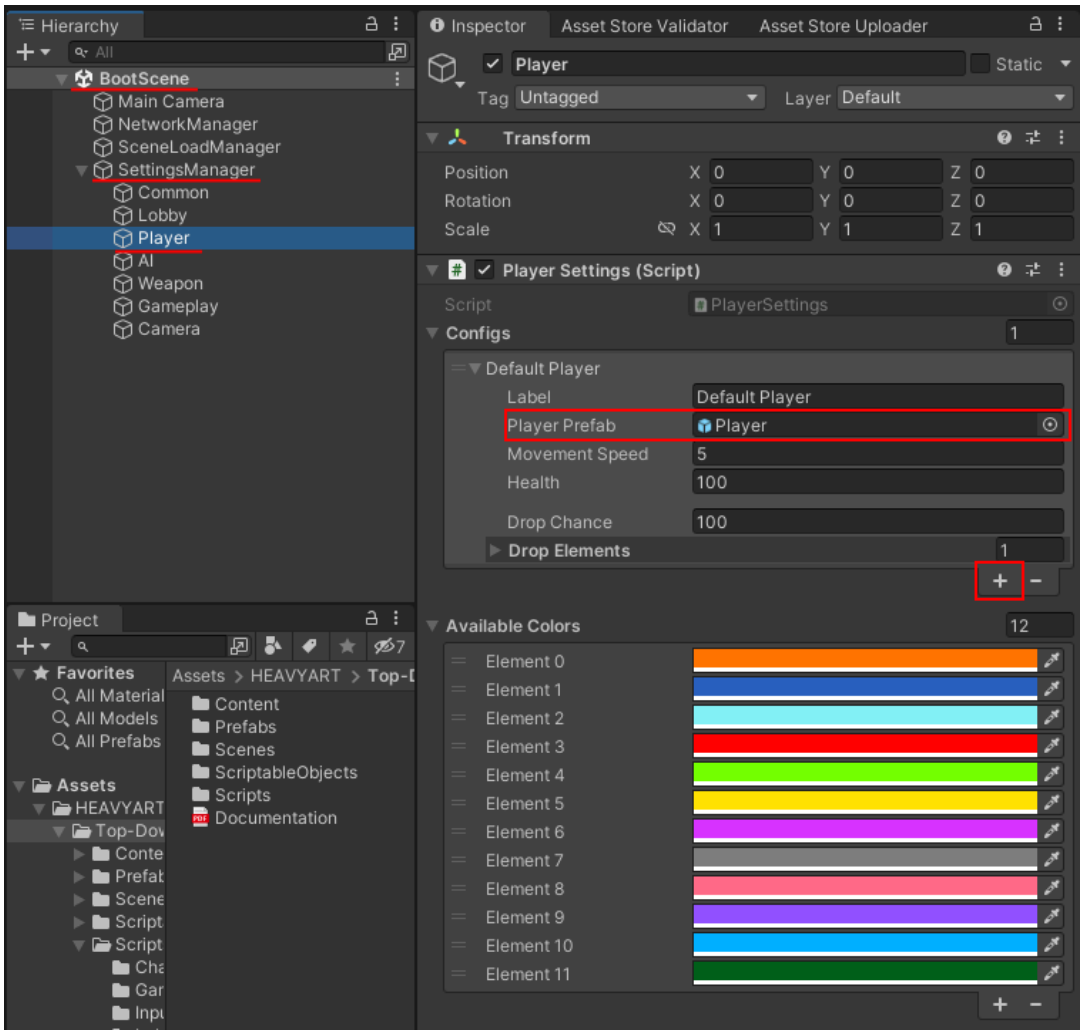
When game spawns a bot, it selects one, randomly from the configs list.
For more details, check source code (NetworkObjectsSpawner.cs).

How to add new player
- Open boot scene.
- Find SettingsManager. Select Player child object.
- Press **+** below Configs collection. It will automatically copy last player config and add it to the end of Configs collection. Now, it's possible to change its settings.

To add new player prefab:
- Copy existing one, and put its copy to Player Prefab field.
- Add new prefab to NetworkPrefabsList located at \ScriptableObjects

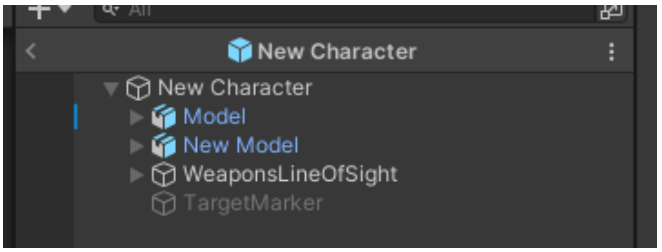
To select a player, set **PlayerDataKeeper.selectedPrefab** value, from code, during runtime.
Currently there is only one player in the project, so value is always 0.
However, it's ready to expand.

For more details, check source code: **PlayerDataKeeper.selectedPrefab** property and **ServiceUserController.GetSerializedPlayerSpawnParameters()** method.

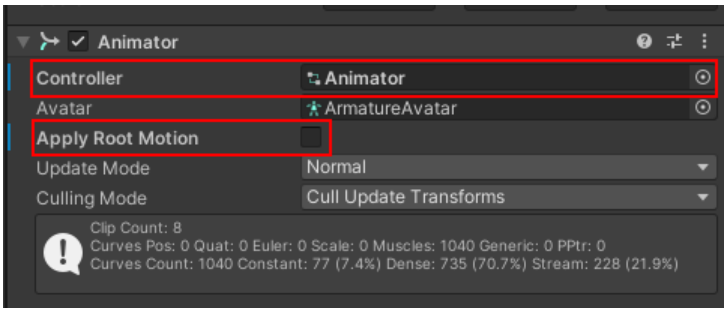


How to change character model (Player, Enemy)

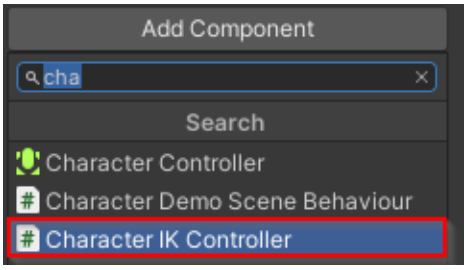
- Open character prefab.
- Put a new character model, next to the old one. Set its position and rotation to zero.



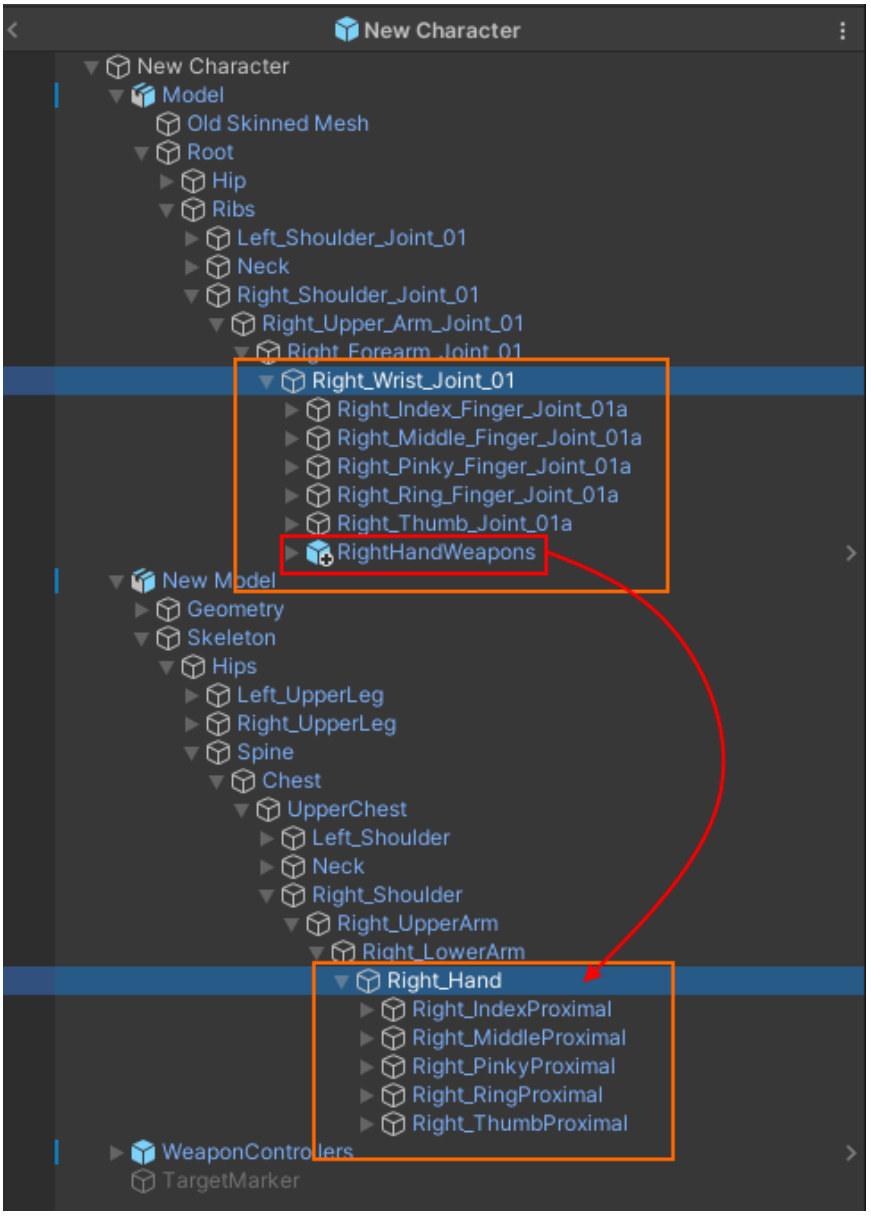
- Copy animator controller from the old model to the new model and turn off root motion.



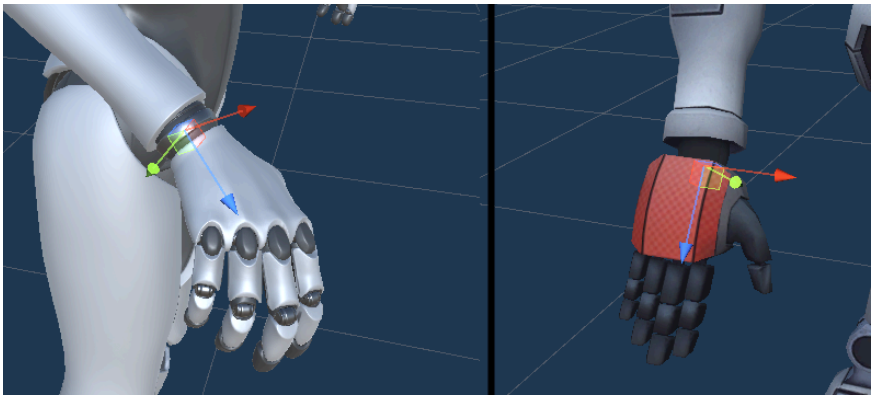
- Add the CharacterIKController component to the new character model. This component will use Inverse Kinematics (IK) to attach model's left hand to the selected weapon's grip.



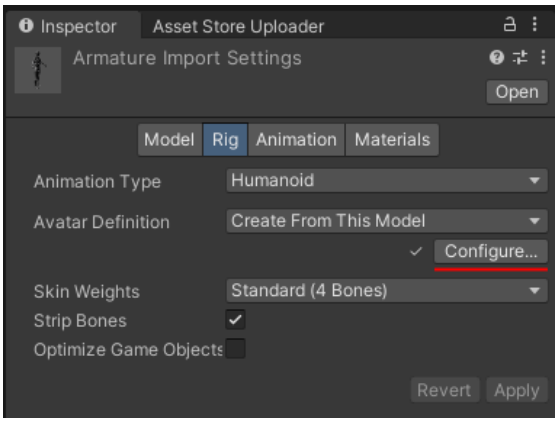
- Drag RightHandWeapons object, from old model's right hand, to new model's right hand:



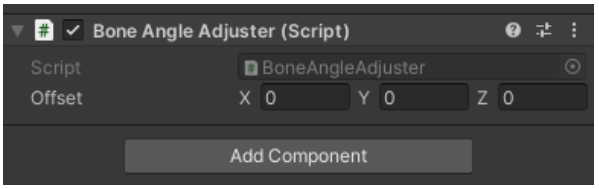
- Set RightHandWeapons position and rotation to zero.
- Make sure RightHandWeapons gizmo pointed in the right direction. Rotate it, if it isn't.



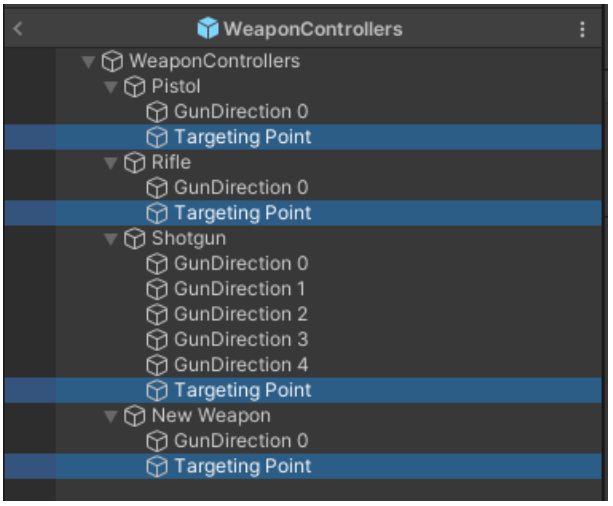
If something still looks wrong, we recommend configuring required bones in the Rig tab on character's model or use **BoneAngleAdjuster** (details below).
Try to rotate required bone in avatar mapping tab and press “Done”. Changes will be applied in runtime.



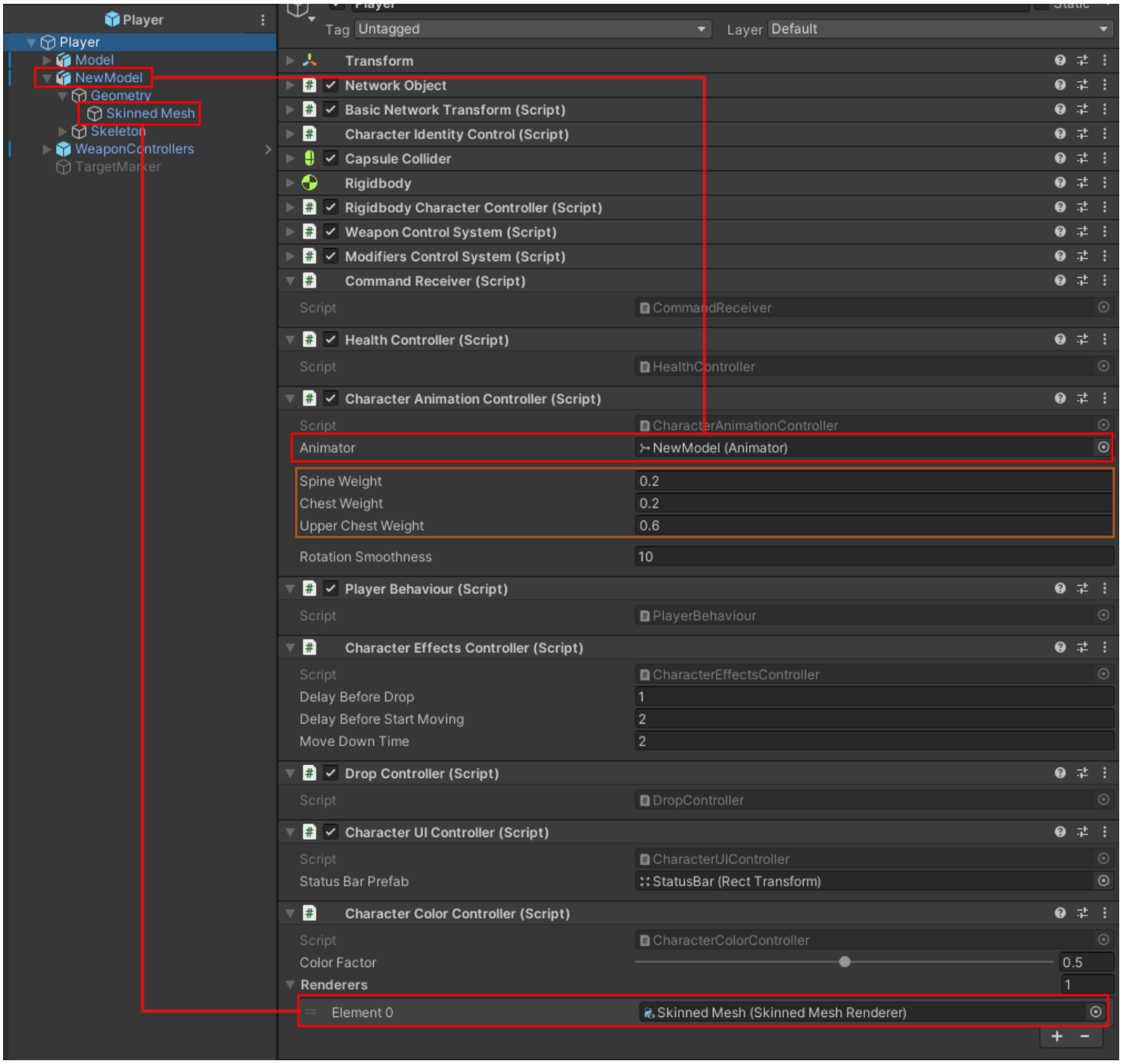
Caution: rig configuration would be saved, even after turning off play mode.
In addition to rig configuration, template includes **BoneAngleAdjuster** - simple tool for adjusting bone rotation in runtime. It’s designed to add offset to existing bone rotation.



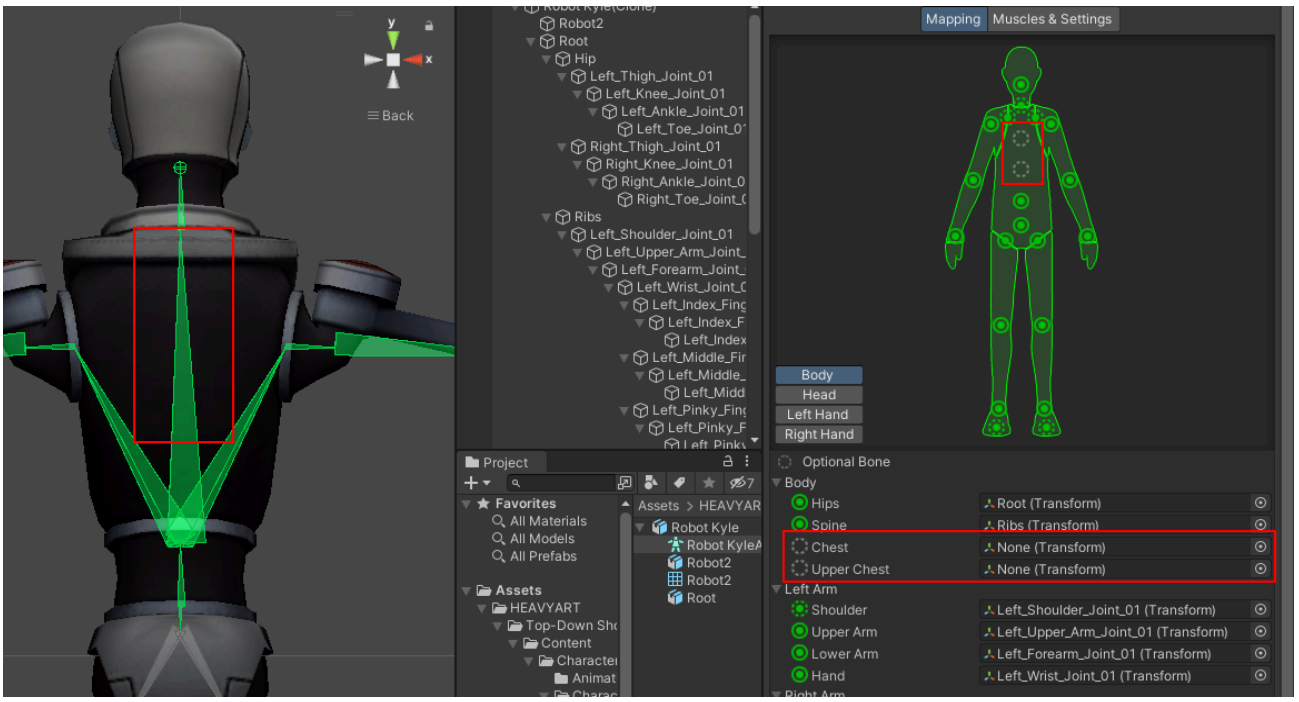
For correction of weapon aiming direction, every weapon controller contains Targeting Point object, parrented to WeaponControllers. It makes it possible to adjust aiming offset for each weapon separately.



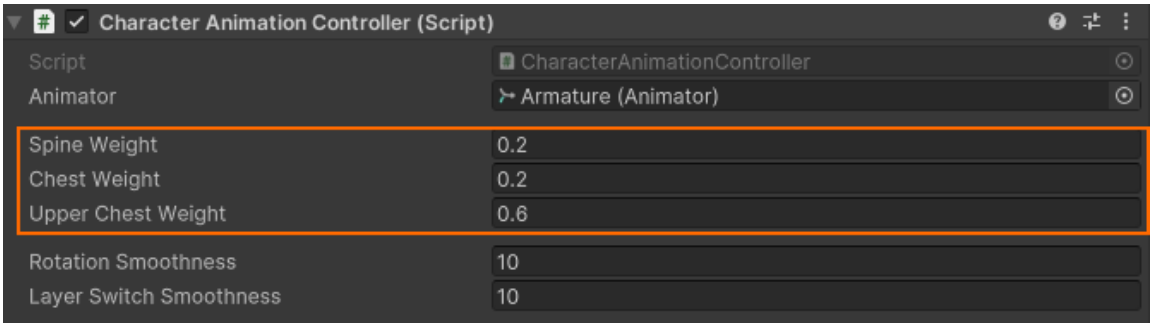
- Update Animator field in Character Animation Controller with new model's animator. (image below)
- If it's a player, update Renderers collection in Character Color Controller with new model's renderers. (AI doesn't have Character Color Controller)
- Set up Spine Weight, Chest Weight and Upper Chest Weight in Character Animation Controller.



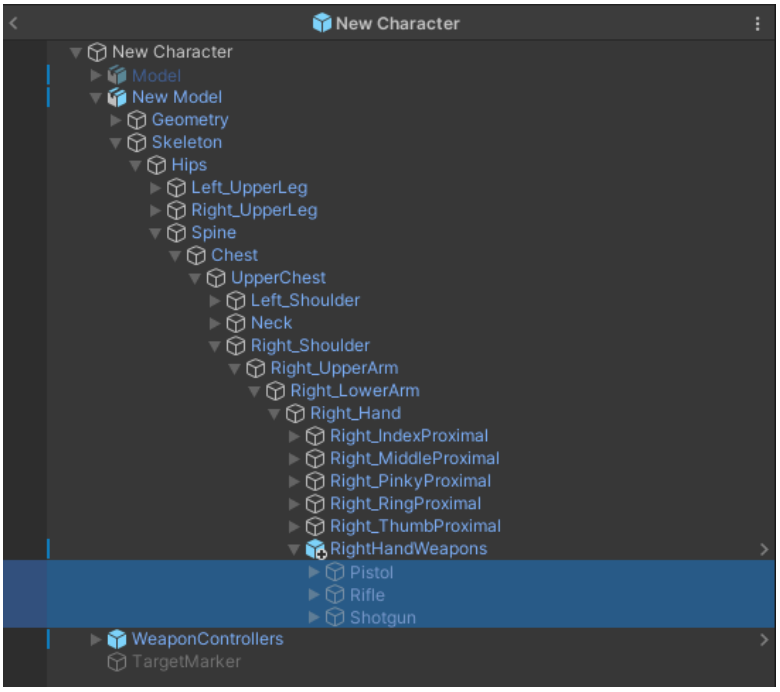
Some skeletons could miss spine bones, like a player's robot model. In this case we recommend to set weights to left existing bones.



Otherwise we recommend to spread weights through all spine bones.

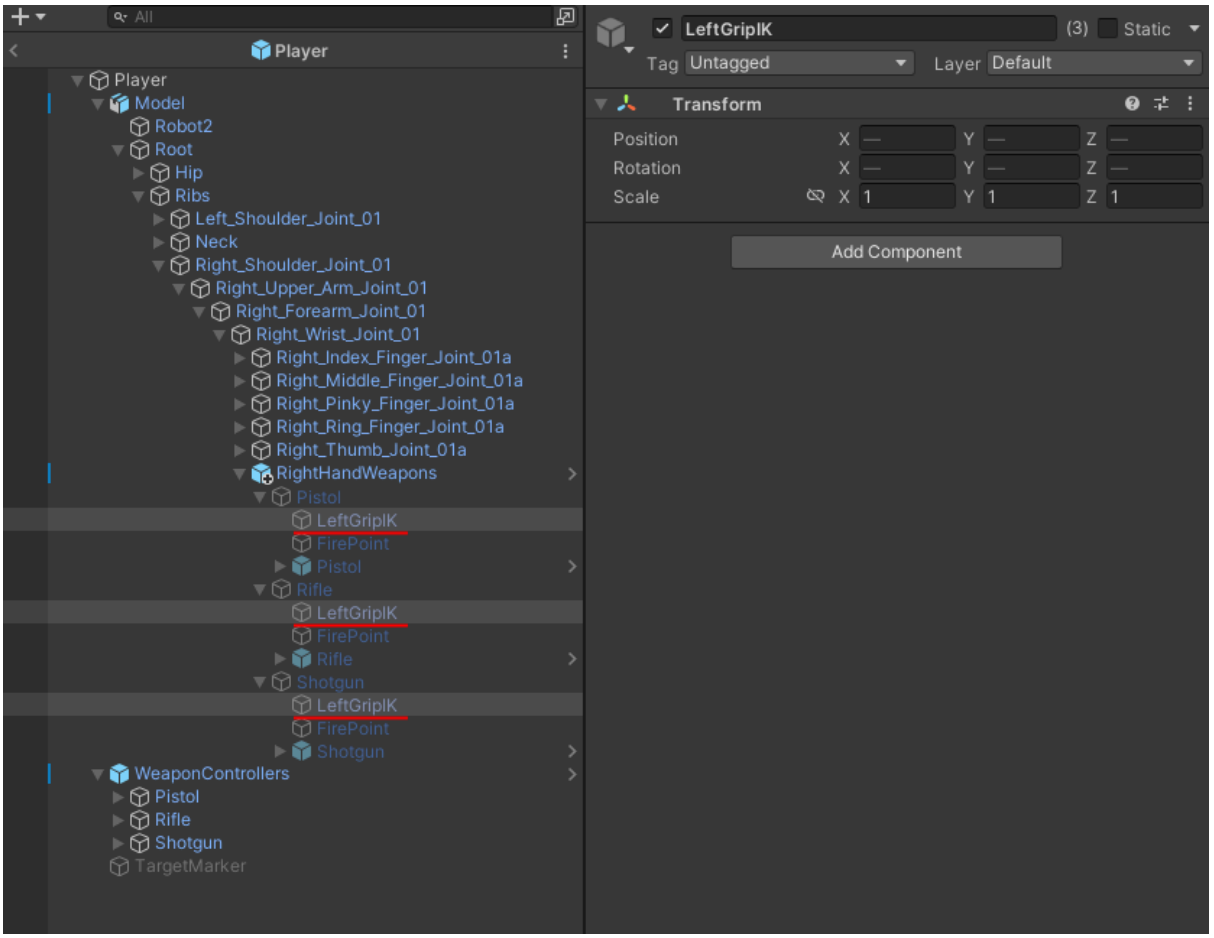


- Adjust positions and rotations for weapon models (located in the right hand).
Note: Don't leave RightHandWeapons child objects turned on, otherwise they will be shown in game, from beginning.

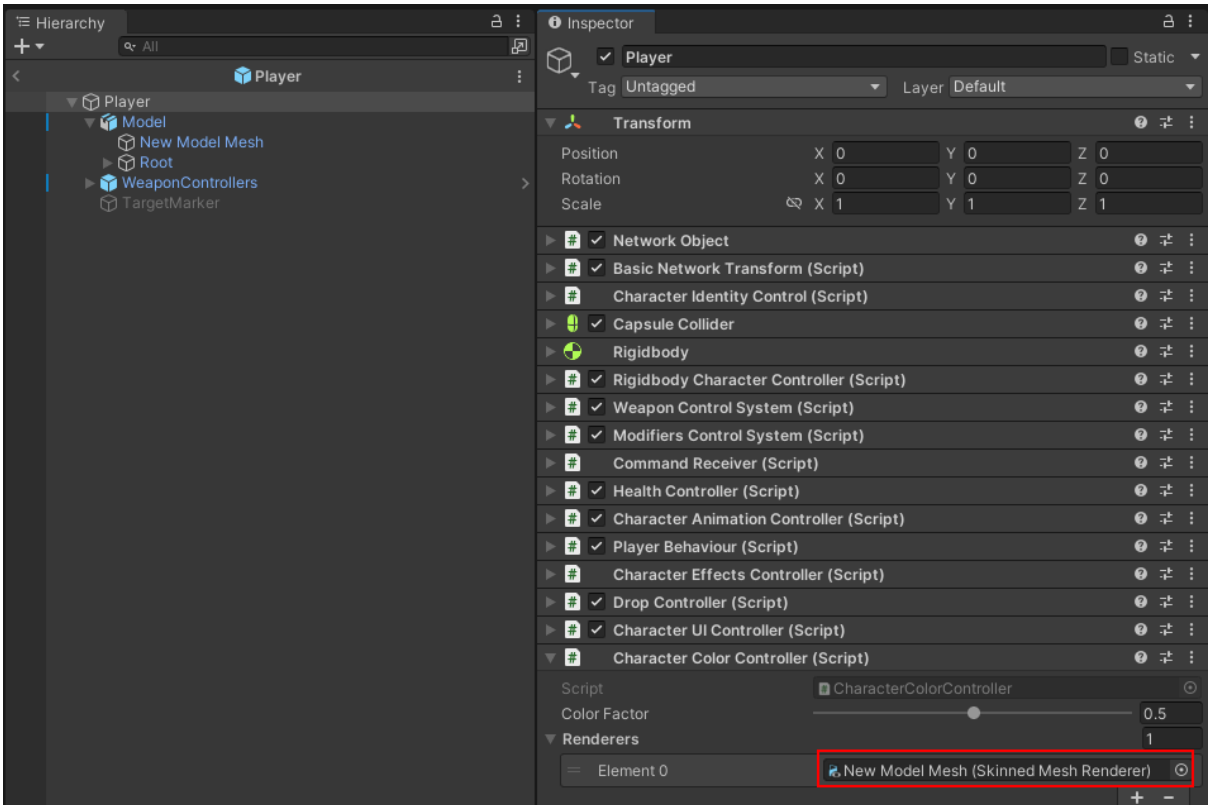


- Now, when everything is done, we can remove the old model and save prefab.

Note: If you need to further adjust model's left hand position, you can tweak LeftGripIK position and rotation for each weapon.

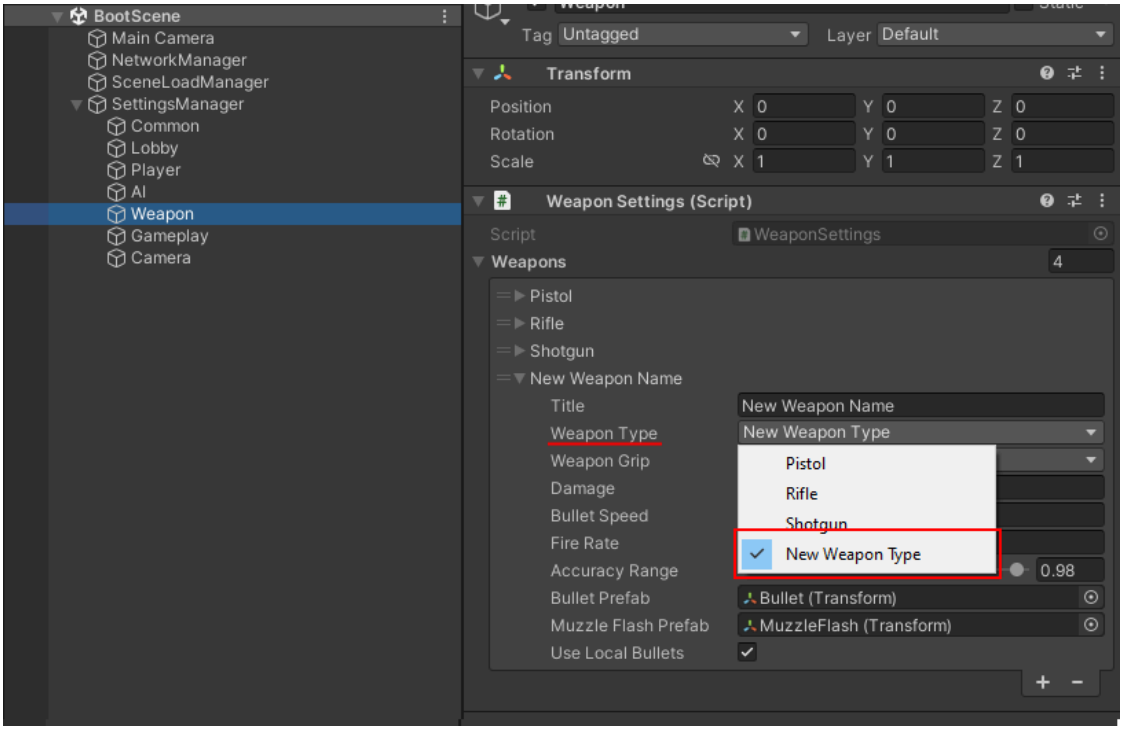


- To make the new model change its color, link character's mesh with **CharacterColorController** component.

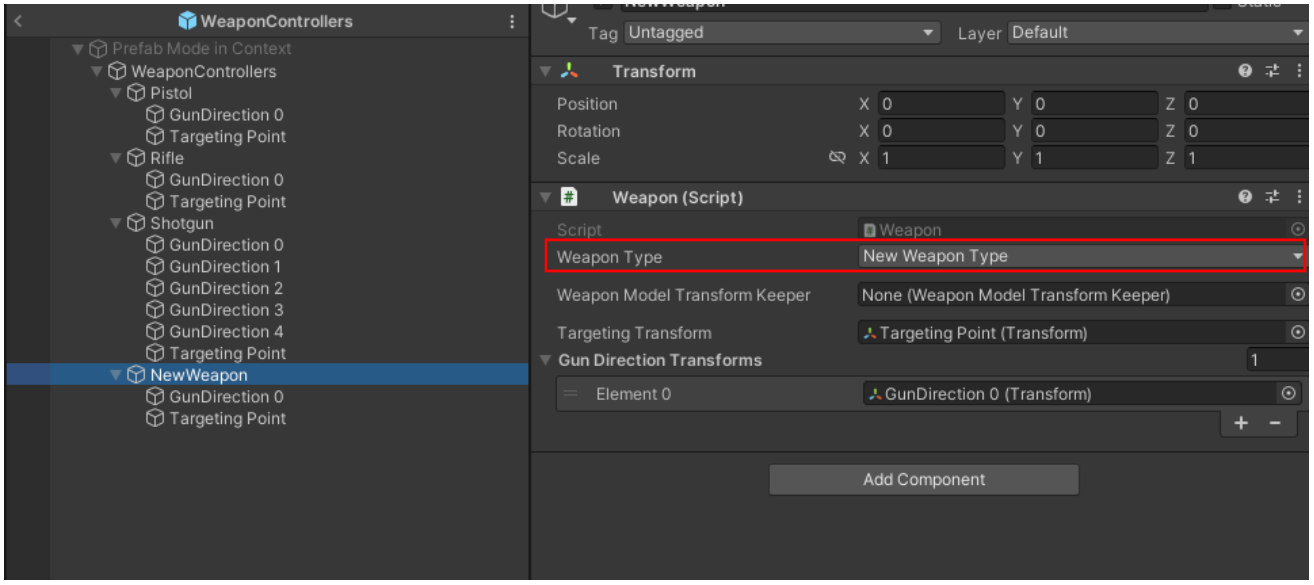


Every **CharacterColorController** contains:
Renderers: reference to the mesh renderers that will be repainted.
All the materials related to the mesh are stored in **SkinnedMeshRenderer**(default mesh component).
Color factor: force of repaint (color replacement). 1 means 100%.

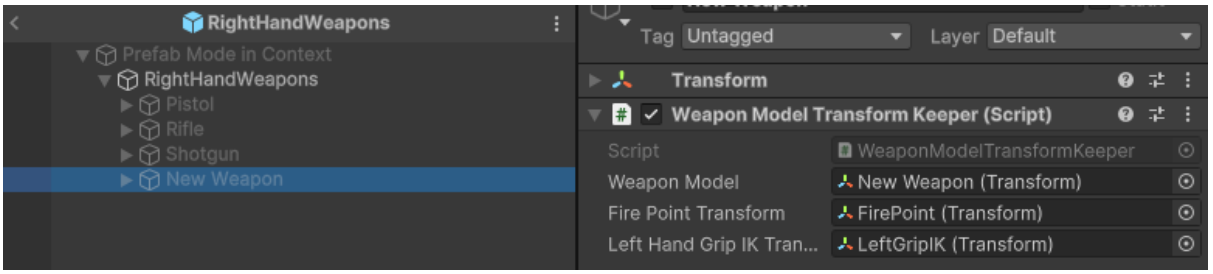
How to add new weapon
- Open WeaponType.cs enum and add new weapon type.
- Open boot scene.
- Find SettingsManager. Select Weapon child object.
- Press **+** below Weapons collection. It will automatically copy last weapon config and add it to the end of Weapons collection.
- Select new defined weapon type in Weapon Type field drop-down menu



- Open WeaponControllers prefab located at \Prefabs\CharacterParts
- Duplicate one of existing weapons and set its Weapon Type to the new type.
Here you can add new guns(barrels) to weapons and set up their default directions.
Usual weapons have one barrel, shotgun has five.
WeaponModelTransformKeeper will be set up inside character prefabs, during next steps.

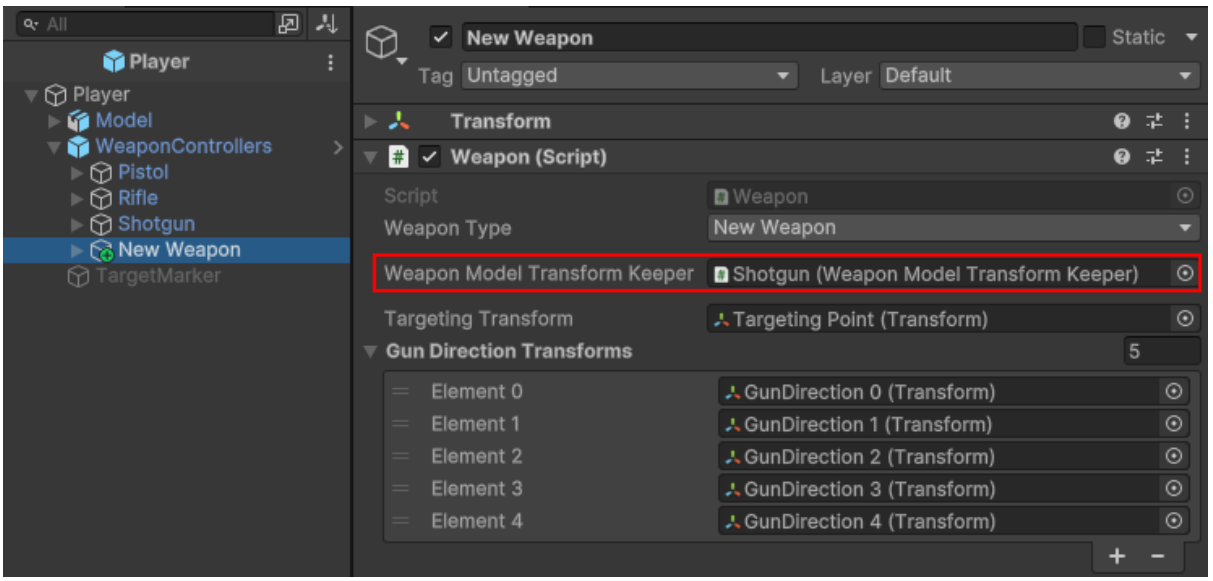


- Open RightHandWeapons prefab located at \Prefabs\CharacterParts
- Duplicate one of RightHandWeapons child objects (weapons).

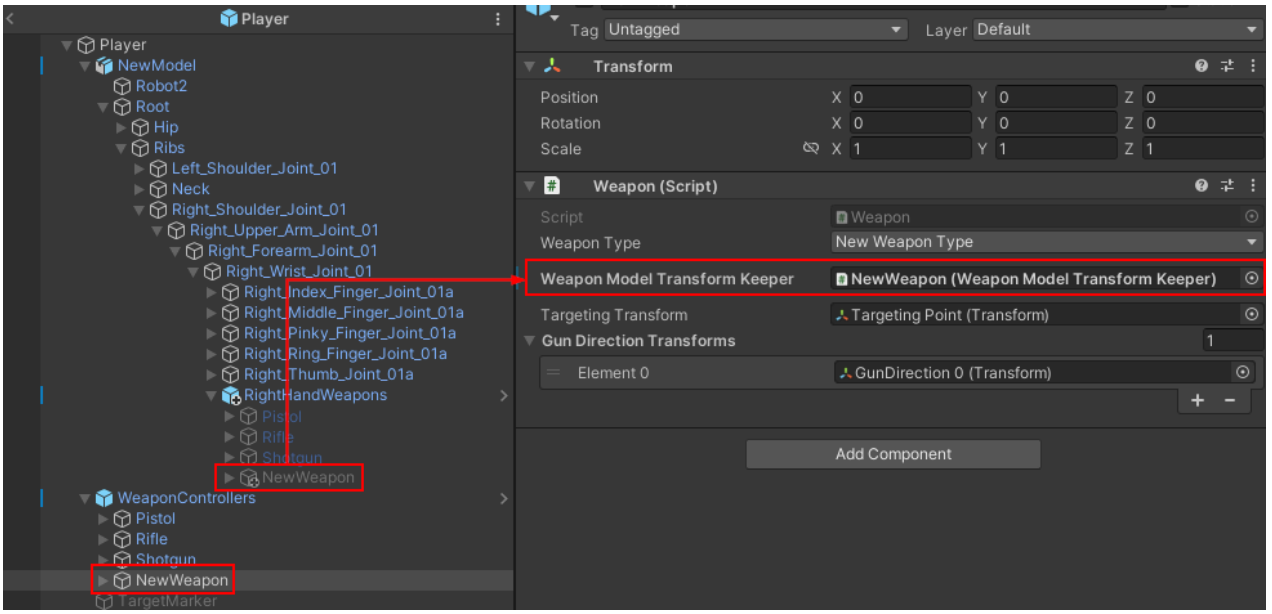


Now it's time to connect newly copied weapon controller to the newly copied weapon model.

- Open character prefab and find RightHandWeapons object inside character's right hand. To locate RightHandWeapons easily, click on any WeaponModelTransformKeeper on any weapon, inside WeaponControllers.

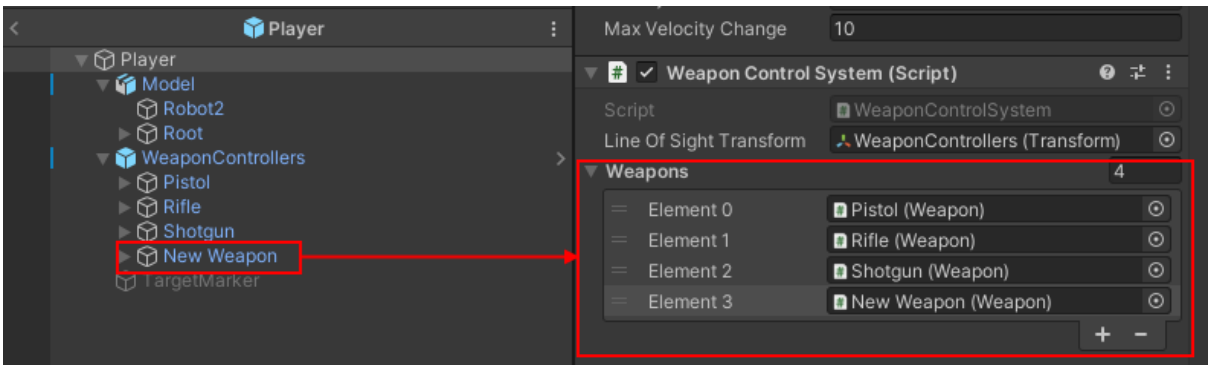


- Link new weapon from RightHandWeapons to the new weapon from WeaponControllers, as shown on screenshot (drag and drop an object).
This is how we let weapons control know where graphics, fire point and left IK grip are located.



Note: Don't leave RightHandWeapons child objects turned on, otherwise they will be shown in game, from beginning.

- Drag and drop the new weapon from WeaponControllers into the Weapons collection in the WeaponControlSystem component on the character's root object.



- Check weapons with debug commands, located in WeaponControlSystem.cs

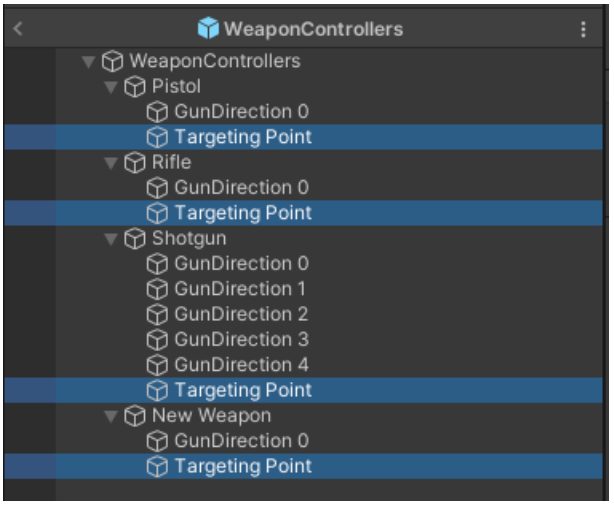
```
Unity Message | 0 references
private void Update()
{
    //Easy weapon switch for debugging
    if (Application.isEditor == true && identityControl.IsLocalPlayer == true)
    {
        if (Input.GetKeyDown(KeyCode.Alpha1)) ActivateWeaponServerRpc(WeaponType.Pistol);

        if (Input.GetKeyDown(KeyCode.Alpha2)) ActivateWeaponServerRpc(WeaponType.Rifle);

        if (Input.GetKeyDown(KeyCode.Alpha3)) ActivateWeaponServerRpc(WeaponType.Shotgun);

        if (Input.GetKeyDown(KeyCode.Alpha4)) ActivateWeaponServerRpc(WeaponType.NewWeaponType);
    }
}
```

Note: For correction of weapon aiming direction, every character contains Targeting Point transform, parrented to WeaponControllers. It makes it possible to adjust aiming offset for every weapon.



How to add another game scene
- Duplicate existing GameScene and rename it. Change anything in Environment object.
- Add scene to Build Settings (otherwise you couldn't run it from code)
- At runtime, before starting a new session, assign **PlayerDataKeeper.selectedScene** with your scene name. Game will remember selected scene.
- Start a session from the main menu UI. Game will use selected scene name automatically.

Currently there is only one scene in the template, so the value is always “GameScene”.

How to add new pick-up object
Pick-ups are made from two parts: Drop Item (prefab) and Scriptable Object with command. Project uses Scriptable Objects as command containers (Damage, Medkit, Weapon Switch) and modifier containers (Speed, Fire Rate, Accuracy).

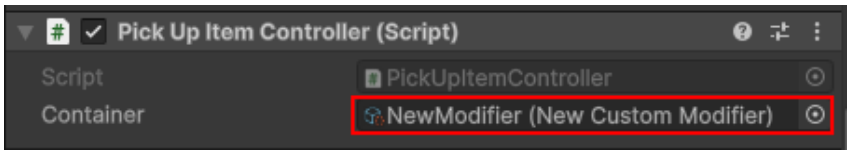
Key differences between commands and modifiers:
Commands designed to run custom logic.
Modifiers designed to be checked in existing components to modify required values (speed, fire rate, accurace, ect) using modifiers internal logic and variables.
Commands inherit from **LogicScriptableObject**, modifiers inherit from **ModifierScriptableObject**.

Active objects of both types will be stored on the character within the **CommandsControlSystem** component.

Existing commands/modifiers	Description
Damage Command	Used as bullet damage. Could be mines, etc
Heal Command	Medkit
Accuracy Modifier	Accuracy power-up with duration
Fire Rate Modifier	Fire rate power-up with duration
Speed Modifier	Movement speed power-up with duration
Weapon Switch Command	Weapon pick-up

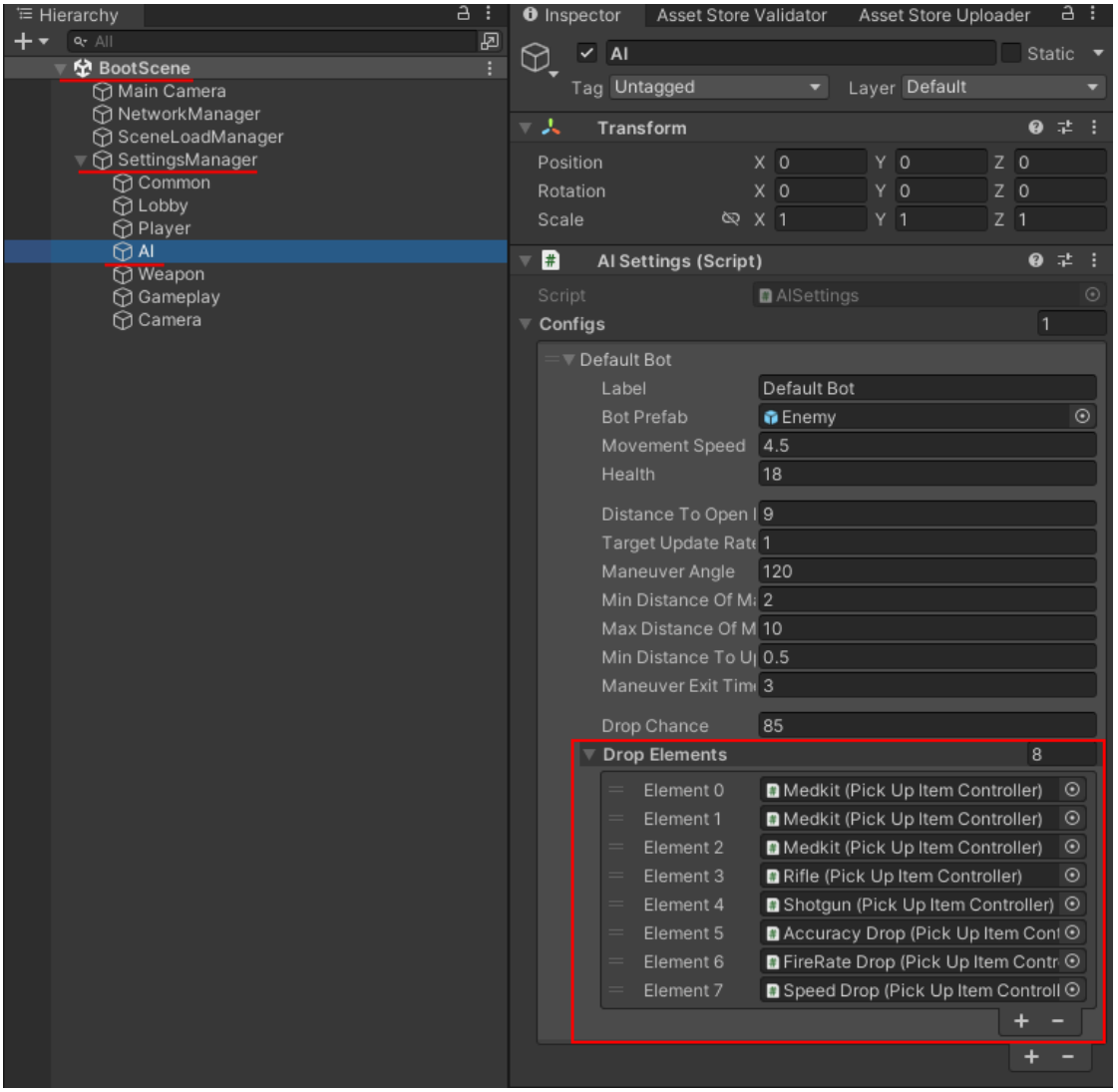
Scriptable objects used with pick-ups located at \ScriptableObjects\Drop Configs.
Pick-up prefabs located at \Prefabs\Drop

To create new pick-up item follow next steps:
- Create Scriptable Object from menu: Assets > Create > Modifier
- Copy one of existing prefabs at \Prefabs\Drop and replace its scriptable object (in Container field) with the newly created scriptable object.



- Open boot scene. Find SettingsManager. Select AI child object.
- Add newly created prefab to AI Settings > Configs > Drop Elements collection.
- Add prefab to Network Prefabs List located at \ScriptableObjects\NetworkPrefabsList.asset

If you want to increase chances for this drop to be instantiated, add it a few times.



How to create new modifier/command type
Project uses Scriptable Objects as command containers (Damage, Medkit, Weapon Switch) and modifier containers (Speed, Fire Rate, Accuracy).
Modifiers and commands designed to be serializable containers of custom logic.

Key differences between commands and modifiers:
Commands designed to run custom logic.
Modifiers designed to be checked in existing components to modify required values (speed, fire rate, accurace, ect) using modifiers internal logic and variables.
Commands inherit from **LogicScriptableObject**, modifiers inherit from **ModifierScriptableObject**.

Active objects of both types will be stored on the character within the **CommandsControlSystem** component.

Note: Deserialized or instantiated Scriptable Objects required to be removed manually, otherwise they will be stored in memory until a new scene will be loaded.
CommandsControlSystem automatically removes all active modifiers and commands after they finish their logic.

To create your own modifiers/commands follow next steps:
- Create a new class and inherit it from **LogicScriptableObject** or **ModifierScriptableObject**.
- Add it to the same namespace as **LogicScriptableObject** and **ModifierScriptableObject.**
- Add CreateAssetMenu attribute (copy below):

[CreateAssetMenu(fileName = "NewCommand", menuName = "Modifier/NewCommand")]

- Override Activate method.
Activate method is the beginning of any command and modifier.
It receives **CommandsControlSystem commandTarget** as an argument.
commandTarget is the character which executes current command.
We can take other components from **commandTarget** using default GetComponent<T>().

- Add custom log to the body of Activate method.
- Add Complete() method call at the end of Activate method.
Complete method will mark our command as finished and ready to remove.

Command code example:

```
[CreateAssetMenu(fileName = "NewCommand", menuName = "Modifier/NewCommand")]
0 references
public class NewCustomModifierContainer : LogicScriptableObject
{
    5 references
    public override void Activate(CommandsControlSystem commandTarget)
    {
        //Add your custom logic here

        Debug.Log("Log from the new command!");

        //Mark command as finished and ready to remove
        Complete();
    }
}
```

Modifiers and commands support public variables visible in inspector.
After adding variables use **[JsonProperty("*custom alias*")]** with each of the variables.
Where "***custom alias***" is a short version of variable name.
Fields without a **[JsonProperty]** attribute will be skipped during serialization.

```
[CreateAssetMenu(fileName = "NewCommand", menuName = "Modifier/NewCommand")]
0 references
public class NewCustomModifierContainer : LogicScriptableObject
{
    //Public visible scriptable object parameters
    [JsonProperty("i")]
    public int newIntParameter;

    [JsonProperty("f")]
    public float newFloatParameter;

    [JsonProperty("s")]
    public string newStringParameter;

    5 references
    public override void Activate(CommandsControlSystem commandTarget)
    {
        //Add your custom logic here

        Debug.Log($"Variables: {newIntParameter} {newFloatParameter} {newStringParameter}");

        //Mark command as finished and ready to remove
        Complete();
    }
}
```

It is possible to call Complete() later. Command will be active for longer and can be used for a few “shots” of custom logic.
In this case we recommend to override FixedUpdate() to work with **Time.time** and internal timers.
Example of modifier with overridden FixedUpdate below.

Modifier example:

```
[CreateAssetMenu(fileName = "NewModifier", menuName = "Modifier/NewModifier")]
Unity Script | 0 references
public class NewCustomModifier : ModifierScriptableObject
{
    [JsonProperty("d")]
    public float duration = 5;
    private float activationTime;

    6 references
    public override void Activate(CommandsControlSystem commandTarget)
    {
        activationTime = Time.time;
    }

    2 references
    public override void FixedUpdate()
    {
        Debug.Log($"Progress: { GetCurrentProgress() }");

        if (Time.time > activationTime + duration)
            Complete();
    }

    2 references
    public override bool TryMergeModifiers(ModifierScriptableObject inputModifier)
    {
        // Add merge custom logic here

        return true;
    }

    4 references
    public override float GetCurrentProgress()
    {
        double timePassed = Time.time - activationTime;
        return (float)(timePassed / duration);
    }
}
```

Modifiers contain two more methods to override: TryMergeModifiers and GetCurrentProgress.

TryMergeModifiers designed to handle the logic for merging two modifiers of the same type. If the method returns **true**, **inputModifier** will be destroyed after execution.

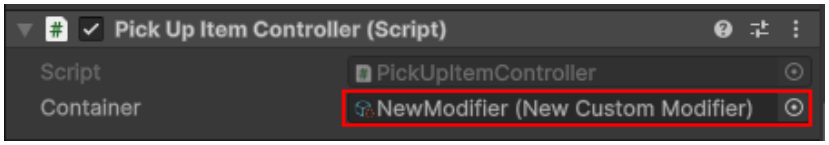
GetCurrentProgress is designed to contain logic of modifier progress based on remaining time. Can contain any other relevant logic.

To optimize the size of serialized modifiers and commands, we recommend registering aliases using the method: **ScriptableObjectJson.RegisterTypeAlias("*alias*", typeof(Type))**. This allows replacing full modifier and command type names with shorter aliases, reducing serialized data size.

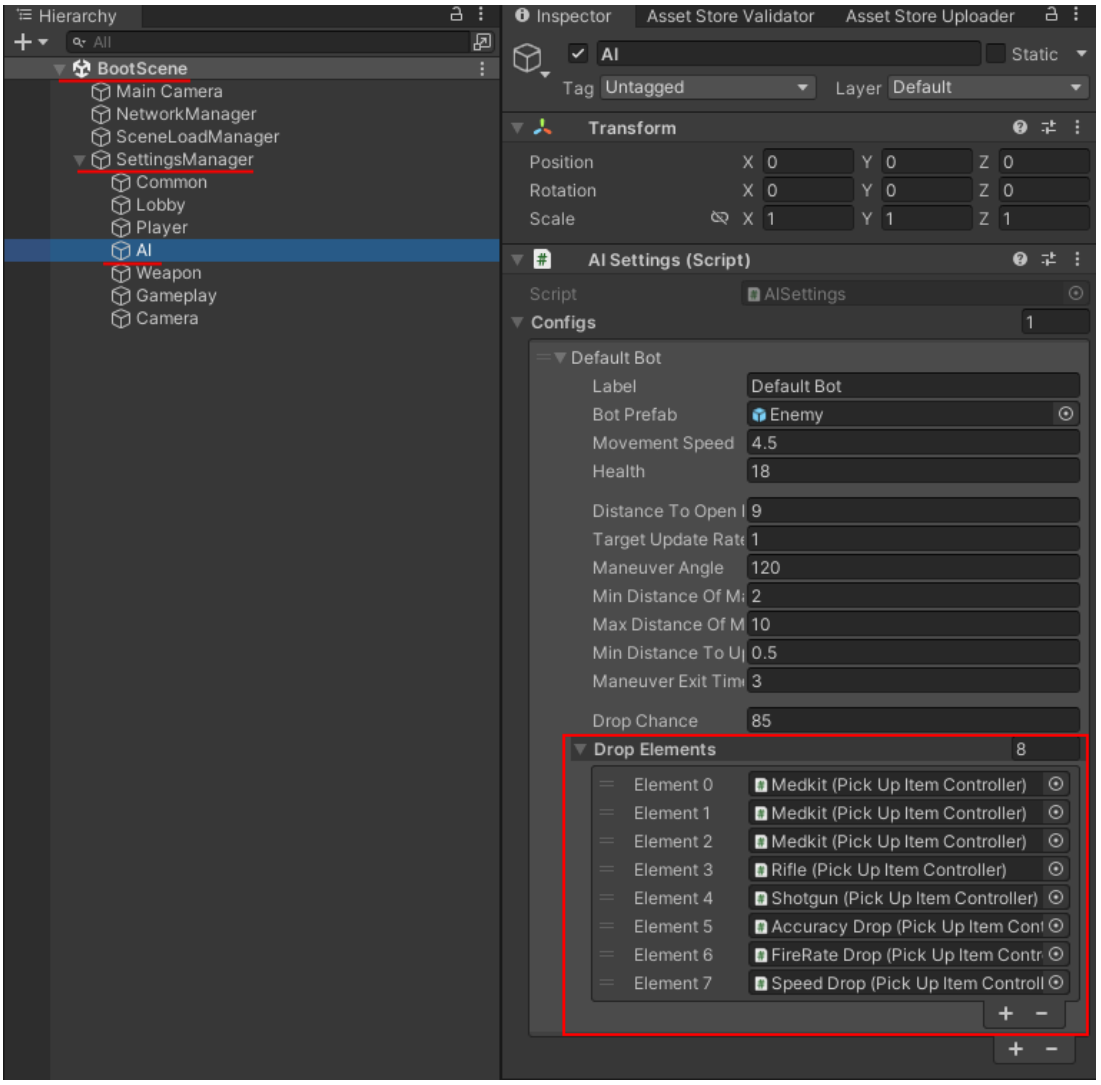
Example:
ScriptableObjectJson.RegisterTypeAlias("HC", typeof(HealCommand));
ScriptableObjectJson.RegisterTypeAlias("DC", typeof(DamageCommand));

Check **SettingsManager.Awake()** method for registered aliases.

- Create Scriptable Object from menu: Assets > Create > Modifier
- Copy one of existing prefabs located at Prefabs\Drop.
- Drag and drop newly created **NewModifier** file into **PickUpItemController** > **Container** field.



- Place prefab on scene or add it to AI settings drop elements list (recommended).



- Add prefab to Network Prefabs List located at **ScriptableObjects\NetworkPrefabsList.asset**
- To use a modifier, add a handling method to the required component that has access to the **CommandsControlSystem**.

Handling method example for new custom modifier:

```
0 references
private float CalculateRelatedMultiplier()
{
    float result = 0;

    for (int i = 0; i < commandsControlSystem.activeCommands.Count; i++)
    {
        //Check modifier by type
        if (commandsControlSystem.activeCommands[i] is NewCustomModifier customModifier)
            result += customModifier.duration;
    }

    return result;
}
```

commandsControlSystem is a reference to the **CommandsControlSystem** component attached to the character's root object.

Handling method example for another modifier:

```
1 reference
private float CalculateSpeedMultiplier()
{
    float resultMultiplier = 1;

    //Handle all the modifiers related to speed
    for (int i = 0; i < commandsControlSystem.activeCommands.Count; i++)
    {
        //Check modifier by type
        if (commandsControlSystem.activeCommands[i] is SpeedModifier speedModifier)
            resultMultiplier *= speedModifier.speedMultiplier;
    }

    return resultMultiplier;
}
```

Can be used in a next way:
float currentSpeed = defaultMovementSpeed * CalculateSpeedMultiplier();

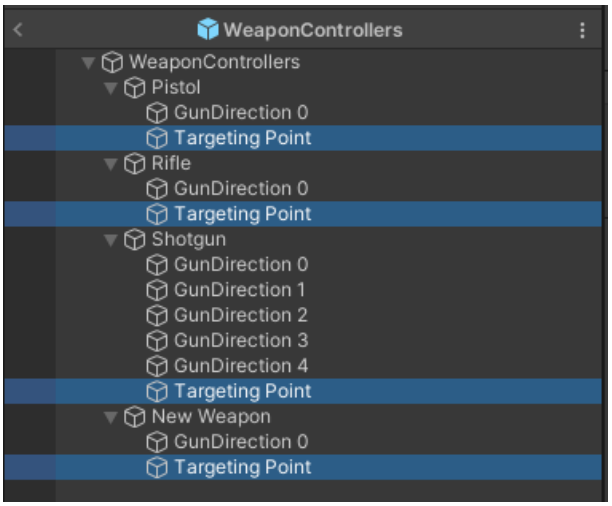
Lobby

Class name	Description
LobbyManager	Singleton. Contains methods to create and join lobby, including private rooms with password. Contains methods to host new game, when lobby is full and ready to start game session.
LobbyDataControl	Part of LobbyManager. Private object. Contains all the necessary properties, events and methods required by lobby to work with data (players, IDs, etc).
LobbyGameHostingControl	Part of LobbyManager. Private object. Contains methods to host or join hosted game using Relay service. Receives join/host commands from LobbyActivityControl. Also contains available regions and its updates.
LobbyActivityControl	Part of LobbyManager. Private object. Contains logic to update lobby timeouts. Required by lobby services, to know, lobby is still alive. Contains logic to check player statuses, before game start. Requires to make sure no one was disconnected (in unhandled way) while waiting, otherwise lobby updates its status to waiting for players.

Game

Class name	Description
GameManager	Singleton. Contains methods and events to control game status. Also contains list of user scores.
NetworkObjectsControl	Part of GameManager. Public object. Stores lists of all characters on scene: players, bots, service objects. Used to have easy access to any type of characters.
NetworkObjectsSpawner	Part of GameManager. Public object. Used to spawn/respawn characters and bots.

Character

Class name	Description
PlayerBehaviour	Playable character logic: movement, rotation, fire command.
AIBehaviour	Bot character logic: navigation, movement, rotation, looking for targets, battle maneuvers, fire command.
BasicNetworkTransform	Simplified logic of NetworkTransform. Easy to set up. No surprises. Contains interpolation and extrapolation.
CharacterNetworkTransform	Advanced version of BasicNetworkTransform. Contains magnitude(movement speed) synchronization for proper movement prediction.
CharacterIdentityControl	Contains identity markers: isPlayer, isBot. Overrides: IsLocalPlayer, IsOwner, OwnerClientId form NetworkBehaviour. Also contains custom character spawn parameters.
RigidbodyCharacterController	Using Unity rigidbody, instead of CharacterController.
CharacterAnimationController	Controls character animations, animation layers and skeleton to aim in the correct direction. For correction of aiming direction, every character contains Targeting Point transform, parrented to WeaponControllers:  It makes it possible to adjust aiming offset for every weapon.
CharacterColorController	Players start color setup.
CharacterDeathController	Currently contains character death scenario (turn off colliders, move under ground etc.)
CharacterUIController	Controls HUD if it's a local player or status bar if it's bot or other player.
DropController	Controls what character could drop on death event.
HealthController	Controls current vitality status. Also runs death event.

TargetMarkerController	Controls marker that shows when someone puts a gun on character.
CommandReceiver	Receives modifiers/commands.
ModifiersControlSystem	Contains currently active power ups (speed, fire rate, accuracy) and commands, waiting for processing (damage, switch weapon). More detailed information is provided in the source code.
WeaponControlSystem	Main weapon control class. Contains all the weapons and controls to manage them. Every weapon has a Weapon component on it. Weapons spawn bullet game objects with Bullet components on them. More detailed information is provided in the source code.

Other

Class name	Description
ServiceUserController	Session user object. For service usage only. Spawns as separated game object.
PickUpItemController	Handles picking up and destroying weapons and power up boxes.
PlayerDataKeeper	Static class. Loads/Saves local data. Works with PlayerPrefs.
SceneLoadManager	Singleton. Works with loading target scenes through LoadingScene. Contains two ways of loading scenes: regular and network oriented. More detailed information is provided in the source code.
GameCameraController	Game camera, targeted on local player.
BoneAngleAdjuster	Small tool to adjust character bone rotation in runtime.